Local search with candidate moves report

Paweł Charkiewicz 160288, Piotr Franc 160306

Objective = cycle length (Euclidean, rounded to nearest integer) + sum of selected node costs. Lower is better.

Pseudocode

All methods read the instance into masterArray, which stores every node (x, y, cost). M = number of runs = 200, N = solution size = ⌈n/2⌉, d(a, b) = Euclidean distance rounded to the nearest integer, s = the current solution (a cycle s[0] → s[1] → ... → s[N − 1] → s[0]), prev(i) / next(i) = the nodes before / after position i on the cycle, Δ = change of the objective (negative = improvement). Every method is the steepest local search from a random starting solution with one intra-route move (NODE_SWAP or EDGE_SWAP) and the inter-route move (exchange of a selected and a not selected node). The baseline evaluates every move of the neighbourhood; the candidate version evaluates only the moves that introduce at least one candidate edge into the solution. All Δ are computed from the changed edges only; no neighbour solution is built to evaluate a move.

Local search (common loop)

```
0. Compute an n × n distance matrix d between every pair of nodes.
1. CANDIDATES only: build the candidate lists once per instance (not included in the running times).
2. Create M random starting solutions (the same seed for every method, so every method starts from
   exactly the same M solutions).
3. For every starting solution s:
4.     Start the timer (it also covers building the starting solution).
5.     Repeat:
6.         pos[v] = position of node v in s, or −1 when v is not selected.
7.         Evaluate the moves (BASELINE: all of them, CANDIDATES: only the candidate moves) and
           remember the one with the smallest Δ (steepest step).
8.     Until the smallest Δ found is ≥ 0, otherwise apply that move and go on.
9.     Stop the timer and save s and its running time = construction time + search time.
```

Candidate lists

```
For every node v:
1. For every other node u compute the key  d(v, u) + cost(u).
2. cand(v) = the K nodes u with the smallest key (ties: the smaller node index first).
```

Candidate moves: only moves that add a candidate edge (a, c)

```
For every position i in s (a = s[i]) and for every c in cand(a):
1. Let j = pos[c].
2. If c is selected (j ≥ 0): two intra-route moves add the edge (a, c)
       EDGE_SWAP: the edge swaps (i, j) and (i − 1, j − 1)
                  (the first adds (s[i], s[j]) and (s[i+1], s[j+1]), the second (s[i−1], s[j−1]) and
                  (s[i], s[j]))
       NODE_SWAP: swap s[j] with s[i + 1] and swap s[j] with s[i − 1] (c lands next to a)
   If c is not selected (j = −1): two inter-route moves add the edge (a, c)
       exchange the node at position i + 1 for c and exchange the node at position i − 1 for c.
3. Positions are taken modulo N. EDGE_SWAP pairs are ordered (p < q) and skipped when q − p < 2 or
   (p, q) = (0, N − 1); NODE_SWAP pairs with p = q are skipped.
Only these moves are generated and evaluated; all other moves are never visited. Exchanging a selected node
with its own nearest neighbour is not a candidate move, because it need not add any candidate edge.
```

Inter-route move: exchange of a selected and a not selected node

```
1. Take position i (node v = s[i], neighbours a = prev(i), b = next(i)) and a not selected node u.
2. Δ = d(a, u) + d(u, b) + cost(u) − d(a, v) − d(v, b) − cost(v)
3. Apply: put u at position i, mark v as not selected and u as selected.
```

NODE_SWAP: two-nodes exchange (intra-route)

```
1. Take positions i < j, nodes x = s[i] and y = s[j].
2. If y comes right after x (… a, x, y, b …):
       Δ = d(a, y) + d(x, b) − d(a, x) − d(y, b)
3. Else if x comes right after y (i = 0, j = N − 1, the cycle wraps: … a, y, x, b …):
       Δ = d(a, x) + d(y, b) − d(a, y) − d(x, b)
4. Otherwise (… a, x, b … c, y, e …):
       Δ = d(a, y) + d(y, b) + d(c, x) + d(x, e) − d(a, x) − d(x, b) − d(c, y) − d(y, e)
5. Apply: swap s[i] and s[j].
```

EDGE_SWAP: two-edges exchange (intra-route)

```
1. Take positions i < j, edges (s[i], s[i+1]) and (s[j], s[j+1]).
2. Δ = d(s[i], s[j]) + d(s[i+1], s[j+1]) − d(s[i], s[i+1]) − d(s[j], s[j+1])
3. Apply: reverse the segment s[i+1] … s[j]; the cycle now uses edges (s[i], s[j]) and (s[i+1], s[j+1]).
```

STEEP: steepest local search (one step)

```
1. Over the evaluated moves (all of them for the baseline, only the candidate moves otherwise):
   remember the move with the smallest Δ; the first one found wins a tie.
2. If that Δ ≥ 0: s is a local optimum of the (candidate) neighbourhood, stop.
3. Otherwise apply the move (an exchange also updates the set of selected nodes).
```

RANDOM: random starting solution

```
1. Start with an empty cycle.
2. Until it holds N nodes: draw a random node index that is not yet used and append it.
```

Randomisation

The steepest search itself is deterministic: it always evaluates the moves in the same order and applies the best one (the first one found wins a tie). The only randomness is in the 200 random starting solutions. Every method restarts the random generator with the same seed (42), so all methods, the baseline and every K, start from exactly the same 200 solutions and the differences between them come from the neighbourhood alone.

Computational experiment

Instances: TSPA, TSPB. Steepest local search from random starting solutions, intra-route move: two-edges exchange and two-nodes exchange; the baseline uses all moves, the candidate versions use K = 5, 10, 15, 20 nearest nodes (by edge length + node cost) per node. Each method was run 200 times. Each cell: average (min – max) over the valid runs; the best average on each instance is in bold.

Objective function

Method	TSPA	TSPB
Steepest LS, two-edges exchange, random start, baseline (all moves)	**76199.32 (73958 – 78983)**	52476.14 (50216 – 55356)
Steepest LS, two-edges exchange, random start, candidate moves K = 5	86375.36 (80050 – 94425)	54045.29 (50600 – 57003)
Steepest LS, two-edges exchange, random start, candidate moves K = 10	80072.75 (75153 – 85444)	52715.43 (50088 – 55330)
Steepest LS, two-edges exchange, random start, candidate moves K = 15	77531.03 (73499 – 81711)	**52463.99 (50315 – 55133)**
Steepest LS, two-edges exchange, random start, candidate moves K = 20	76455.44 (73826 – 80577)	52483.69 (50381 – 55431)
Steepest LS, two-nodes exchange, random start, baseline (all moves)	90174.50 (84195 – 98981)	65740.75 (58746 – 73286)
Steepest LS, two-nodes exchange, random start, candidate moves K = 5	106085.21 (97087 – 116131)	70953.60 (63744 – 83507)
Steepest LS, two-nodes exchange, random start, candidate moves K = 10	96429.09 (86851 – 109373)	67115.61 (59665 – 75741)
Steepest LS, two-nodes exchange, random start, candidate moves K = 15	92829.99 (85620 – 102307)	66635.66 (58873 – 73447)
Steepest LS, two-nodes exchange, random start, candidate moves K = 20	91539.32 (84345 – 100958)	65917.39 (59787 – 76385)

Running time [ms]

Time of one run: building its starting solution + the local search. The candidate lists are built once per instance and are not included.

Method	TSPA	TSPB
Steepest LS, two-edges exchange, random start, baseline (all moves)	21.297 (17.946 – 40.820)	20.400 (18.207 – 26.037)
Steepest LS, two-edges exchange, random start, candidate moves K = 5	**2.206 (1.625 – 5.519)**	**1.883 (1.592 – 2.251)**
Steepest LS, two-edges exchange, random start, candidate moves K = 10	3.742 (3.398 – 4.240)	3.802 (3.350 – 5.734)
Steepest LS, two-edges exchange, random start, candidate moves K = 15	5.781 (5.092 – 14.766)	5.733 (4.970 – 8.266)
Steepest LS, two-edges exchange, random start, candidate moves K = 20	7.809 (6.782 – 9.124)	7.642 (6.619 – 9.105)
Steepest LS, two-nodes exchange, random start, baseline (all moves)	32.172 (25.766 – 54.896)	31.469 (23.897 – 43.364)
Steepest LS, two-nodes exchange, random start, candidate moves K = 5	3.256 (2.245 – 4.349)	3.217 (2.045 – 4.267)
Steepest LS, two-nodes exchange, random start, candidate moves K = 10	7.155 (5.409 – 11.634)	6.993 (5.500 – 9.390)

Method	TSPA	TSPB
Steepest LS, two-nodes exchange, random start, candidate moves K = 15	11.242 (9.065 – 15.404)	10.458 (7.906 – 13.590)
Steepest LS, two-nodes exchange, random start, candidate moves K = 20	14.962 (10.792 – 24.805)	14.363 (11.176 – 22.447)

Candidate moves versus the baseline

Speed-up = average baseline time / average time of the method; objective = relative change of the average objective against the baseline (positive = worse).

Method	TSPA speed-up	TSPA objective	TSPB speed-up	TSPB objective
Steepest LS, two-edges exchange, random start, candidate moves K = 5	9.65×	+13.35 %	10.83×	+2.99 %
Steepest LS, two-edges exchange, random start, candidate moves K = 10	5.69×	+5.08 %	5.37×	+0.46 %
Steepest LS, two-edges exchange, random start, candidate moves K = 15	3.68×	+1.75 %	3.56×	-0.02 %
Steepest LS, two-edges exchange, random start, candidate moves K = 20	2.73×	+0.34 %	2.67×	+0.01 %
Steepest LS, two-nodes exchange, random start, candidate moves K = 5	9.88×	+17.64 %	9.78×	+7.93 %
Steepest LS, two-nodes exchange, random start, candidate moves K = 10	4.50×	+6.94 %	4.50×	+2.09 %
Steepest LS, two-nodes exchange, random start, candidate moves K = 15	2.86×	+2.94 %	3.01×	+1.36 %
Steepest LS, two-nodes exchange, random start, candidate moves K = 20	2.15×	+1.51 %	2.19×	+0.27 %

Validation

Valid = the run has ⌈n/2⌉ distinct nodes of the instance. Local optima = no improving move remains in the method's own neighbourhood (all moves for the baseline, only the candidate moves for K = ...; checked independently by the report script; "best Δ" is the most improving move left). Full nbhd. = how many runs are also local optima of the full neighbourhood (informational).

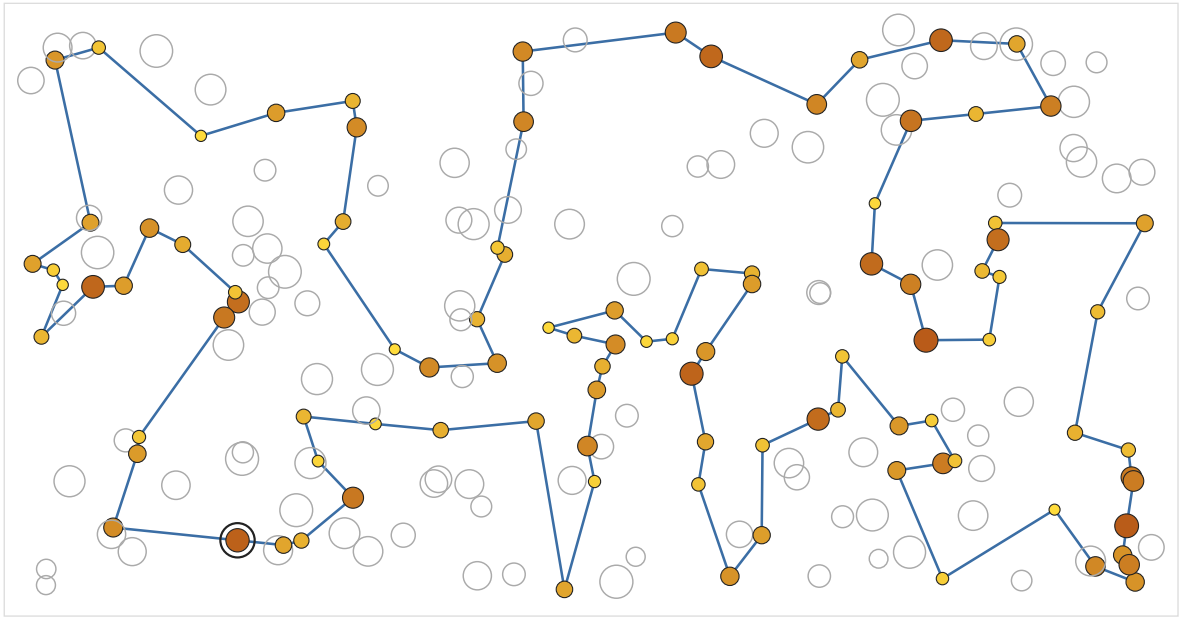
Method	TSPA valid	TSPA local optima	TSPA full nbhd.	TSPB valid	TSPB local optima	TSPB full nbhd.
Steepest LS, two-edges exchange, random start, baseline (all moves)	**200 / 200**	**200 / 200**	200 / 200	**200 / 200**	**200 / 200**	200 / 200
Steepest LS, two-edges exchange, random start, candidate moves K = 5	**200 / 200**	**200 / 200**	0 / 200	**200 / 200**	**200 / 200**	0 / 200
Steepest LS, two-edges exchange, random start, candidate moves K = 10	**200 / 200**	**200 / 200**	0 / 200	**200 / 200**	**200 / 200**	55 / 200
Steepest LS, two-edges exchange, random start, candidate moves K = 15	**200 / 200**	**200 / 200**	3 / 200	**200 / 200**	**200 / 200**	130 / 200
Steepest LS, two-edges exchange, random start, candidate moves K = 20	**200 / 200**	**200 / 200**	78 / 200	**200 / 200**	**200 / 200**	176 / 200
Steepest LS, two-nodes exchange, random start, baseline (all moves)	**200 / 200**	**200 / 200**	200 / 200	**200 / 200**	**200 / 200**	200 / 200
Steepest LS, two-nodes exchange, random start, candidate moves K = 5	**200 / 200**	**200 / 200**	0 / 200	**200 / 200**	**200 / 200**	0 / 200
Steepest LS, two-nodes exchange, random start, candidate moves K = 10	**200 / 200**	**200 / 200**	0 / 200	**200 / 200**	**200 / 200**	4 / 200
Steepest LS, two-nodes exchange, random start, candidate moves K = 15	**200 / 200**	**200 / 200**	0 / 200	**200 / 200**	**200 / 200**	29 / 200
Steepest LS, two-nodes exchange, random start, candidate moves K = 20	**200 / 200**	**200 / 200**	12 / 200	**200 / 200**	**200 / 200**	71 / 200

All runs valid and all runs are local optima of their own neighbourhood.

Instance TSPA

Steepest LS, two-edges exchange, random start, baseline (all moves)

Best of 200 valid runs: run #59 with objective **73958**.



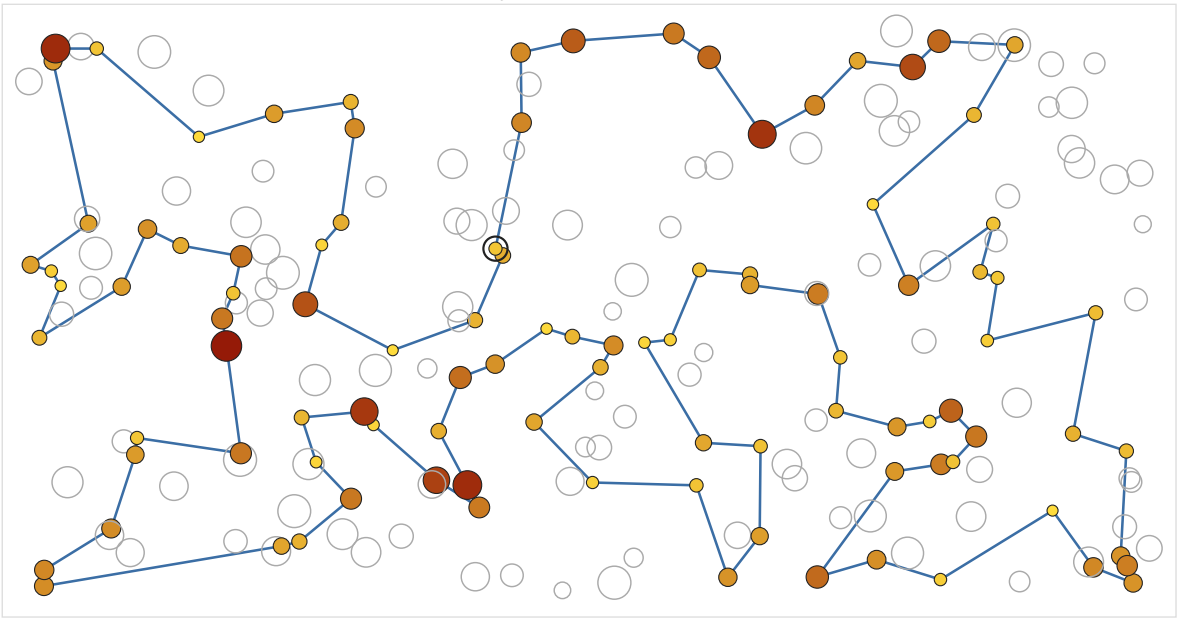
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
92, 69, 127, 97, 160, 83, 123, 152, 108, 49, 11, 168, 63, 149, 116, 60, 31, 48, 80, 91, 157, 71, 76, 132, 180,
28, 185, 35, 16, 51, 112, 115, 39, 154, 186, 136, 163, 29, 101, 178, 184, 164, 142, 85, 81, 93, 50, 3, 169, 1,
147, 47, 130, 159, 146, 170, 75, 153, 193, 78, 88, 107, 57, 151, 42, 89, 106, 150, 174, 176, 72, 134, 158, 192,
58, 84, 99, 131, 54, 9, 33, 36, 177, 52, 67, 144, 179, 53, 140, 73, 135, 114, 45, 87, 141, 30, 187, 189, 2, 171
```

Steepest LS, two-edges exchange, random start, candidate moves K = 5

Best of 200 valid runs: run #92 with objective **80050**.



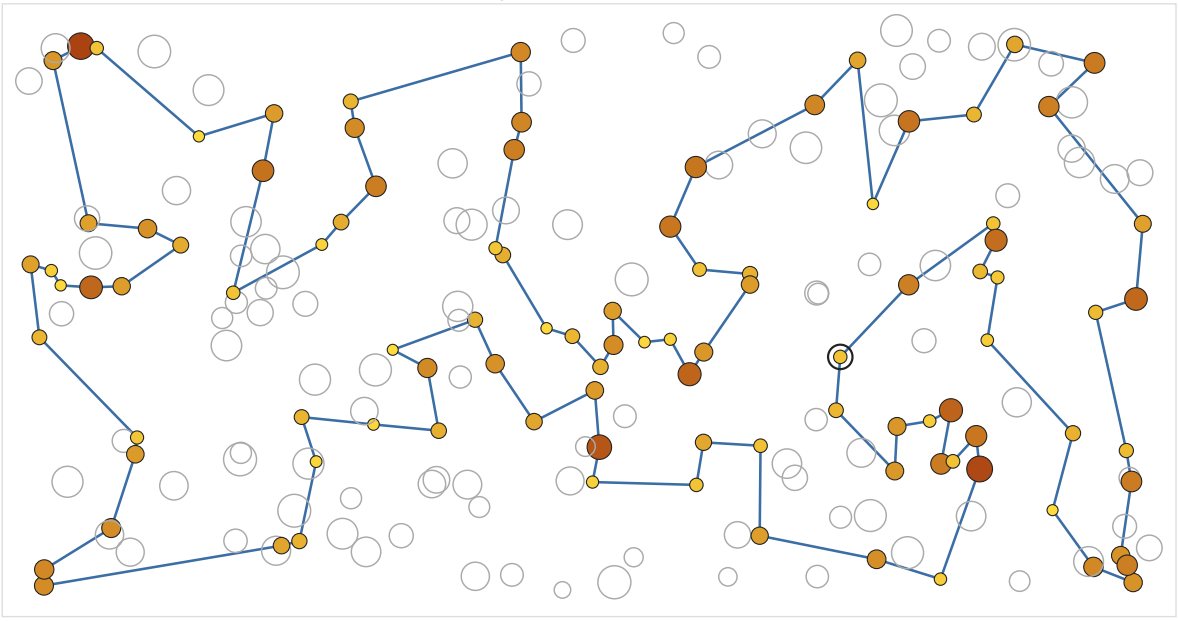
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
51, 112, 115, 175, 39, 154, 34, 186, 136, 24, 163, 29, 178, 164, 85, 1, 3, 50, 93, 47, 130, 159, 153, 193, 78,
88, 107, 57, 182, 111, 151, 42, 89, 113, 43, 106, 150, 176, 174, 8, 54, 9, 33, 36, 177, 84, 134, 158, 192, 58,
135, 45, 53, 179, 144, 67, 185, 102, 87, 70, 98, 27, 141, 145, 30, 187, 189, 2, 171, 104, 129, 69, 127, 97, 161,
12, 160, 123, 100, 152, 108, 49, 168, 63, 149, 116, 60, 31, 199, 48, 80, 91, 157, 71, 76, 132, 162, 180, 35, 16
```

Steepest LS, two-edges exchange, random start, candidate moves K = 10

Best of 200 valid runs: run #14 with objective **75153**.



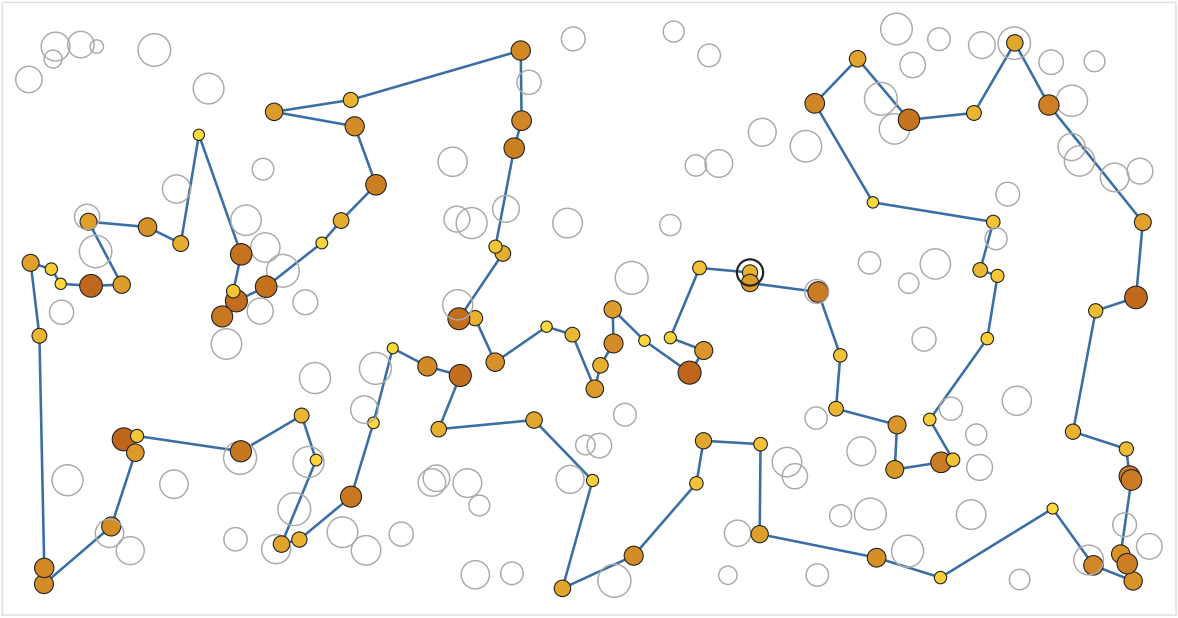
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
174, 85, 1, 169, 3, 50, 93, 130, 107, 88, 78, 193, 153, 170, 159, 47, 37, 147, 101, 55, 29, 178, 184, 164, 136,
186, 62, 143, 33, 9, 54, 131, 99, 36, 177, 52, 179, 53, 144, 67, 16, 51, 139, 112, 115, 157, 71, 4, 76, 132,
123, 38, 91, 80, 48, 7, 31, 60, 108, 152, 49, 11, 63, 149, 116, 168, 97, 127, 69, 129, 104, 171, 2, 187, 30,
141, 87, 28, 180, 35, 185, 45, 140, 61, 135, 58, 84, 134, 158, 182, 57, 103, 113, 89, 42, 43, 106, 150, 151, 176
```

Steepest LS, two-edges exchange, random start, candidate moves K = 15

Best of 200 valid runs: run #149 with objective **73499**.



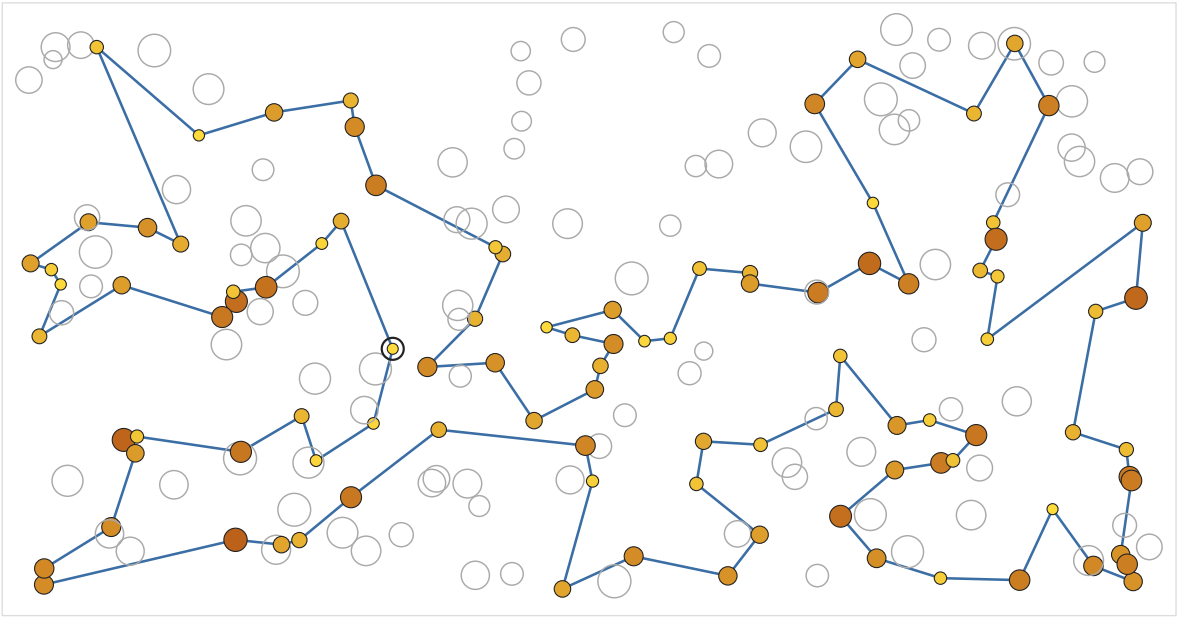
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
9, 33, 36, 131, 99, 177, 52, 179, 53, 140, 144, 67, 185, 35, 156, 16, 51, 139, 112, 115, 157, 91, 71, 4, 76,
132, 90, 83, 160, 123, 100, 80, 152, 108, 60, 49, 11, 63, 149, 116, 168, 129, 104, 69, 127, 95, 97, 161, 30,
187, 171, 2, 189, 141, 180, 28, 102, 87, 45, 135, 114, 181, 58, 84, 134, 158, 182, 57, 107, 88, 78, 193, 153,
170, 146, 159, 130, 47, 37, 147, 101, 29, 178, 184, 136, 186, 164, 1, 3, 50, 93, 106, 89, 42, 151, 150, 176,
174, 8, 54
```

Steepest LS, two-edges exchange, random start, candidate moves K = 20

Best of 200 valid runs: run #198 with objective **73826**.



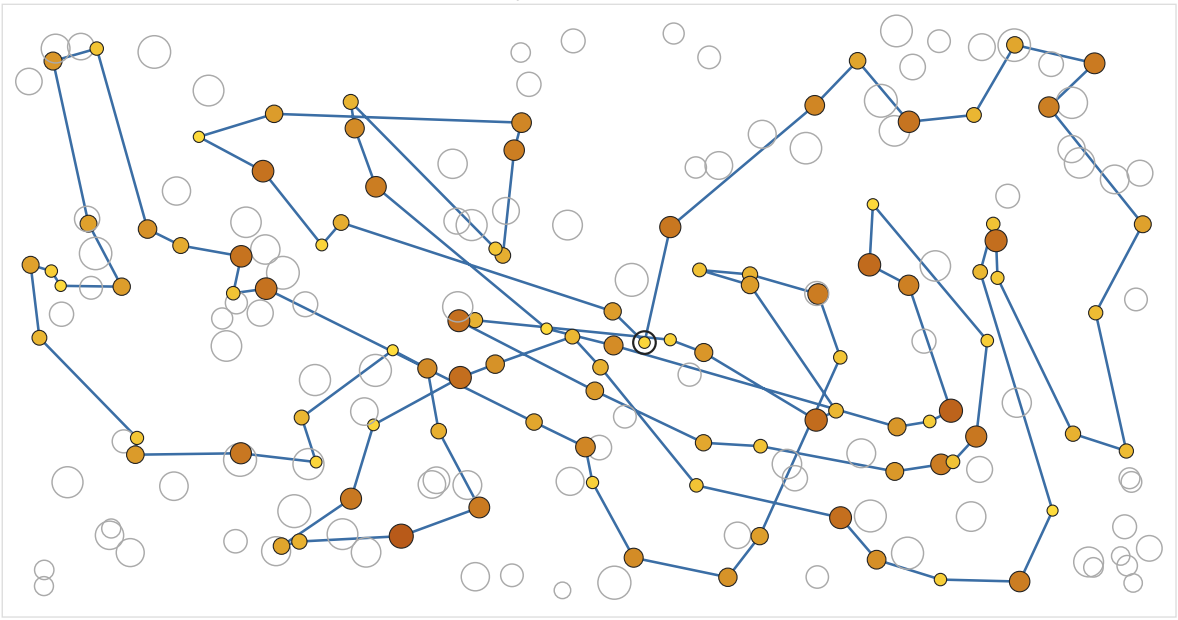
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
180, 141, 187, 30, 161, 97, 95, 127, 69, 129, 104, 92, 171, 2, 189, 87, 73, 135, 114, 181, 192, 158, 58, 84,
134, 176, 174, 150, 106, 113, 89, 42, 151, 138, 182, 57, 64, 107, 88, 78, 193, 153, 170, 146, 159, 130, 47, 37,
147, 93, 50, 3, 169, 1, 101, 29, 178, 136, 186, 164, 85, 142, 8, 54, 9, 33, 36, 177, 52, 67, 144, 179, 53, 140,
45, 185, 28, 35, 16, 51, 4, 71, 157, 91, 80, 48, 152, 108, 60, 116, 149, 63, 168, 49, 160, 83, 123, 90, 132, 76
```

Steepest LS, two-nodes exchange, random start, baseline (all moves)

Best of 200 valid runs: run #163 with objective **84195**.



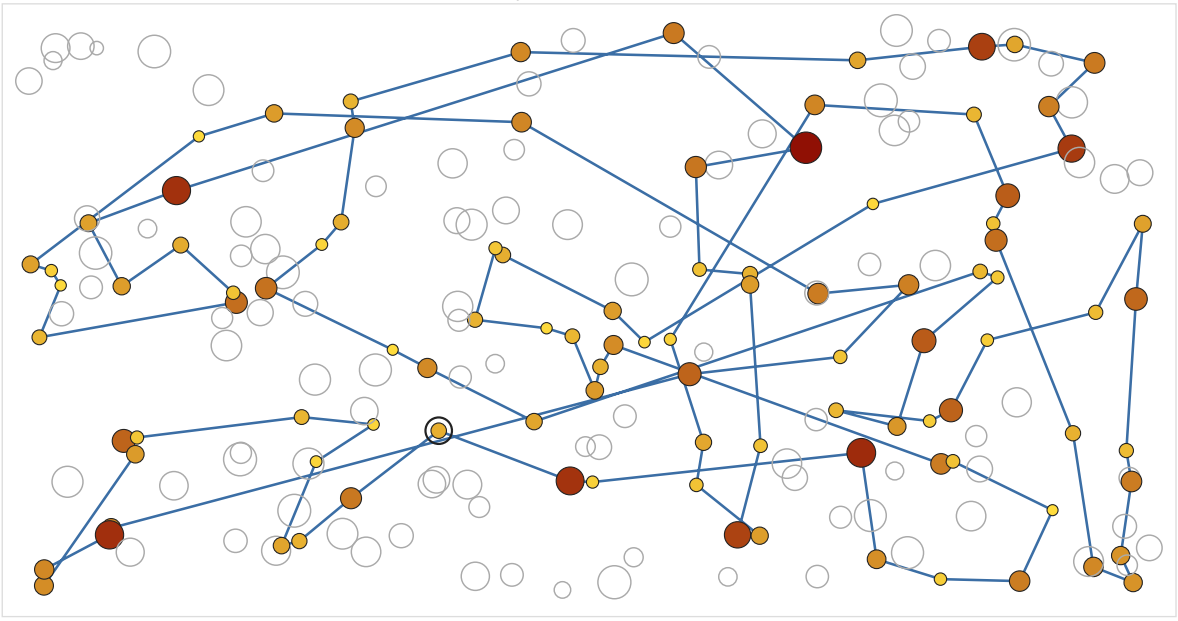
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
177, 143, 186, 136, 184, 178, 29, 55, 101, 147, 47, 159, 130, 50, 169, 1, 3, 107, 64, 57, 182, 138, 58, 53, 144,
185, 102, 141, 189, 171, 2, 10, 98, 87, 28, 180, 30, 187, 161, 127, 97, 168, 116, 149, 63, 49, 60, 31, 48, 108,
152, 100, 123, 90, 45, 73, 135, 181, 192, 158, 174, 8, 9, 33, 54, 176, 72, 131, 36, 35, 156, 140, 84, 134, 151,
42, 89, 113, 93, 164, 142, 85, 43, 106, 150, 179, 67, 4, 71, 157, 51, 16, 139, 112, 91, 80, 38, 132, 76, 52
```

Steepest LS, two-nodes exchange, random start, candidate moves K = 5

Best of 200 valid runs: run #129 with objective **97087**.



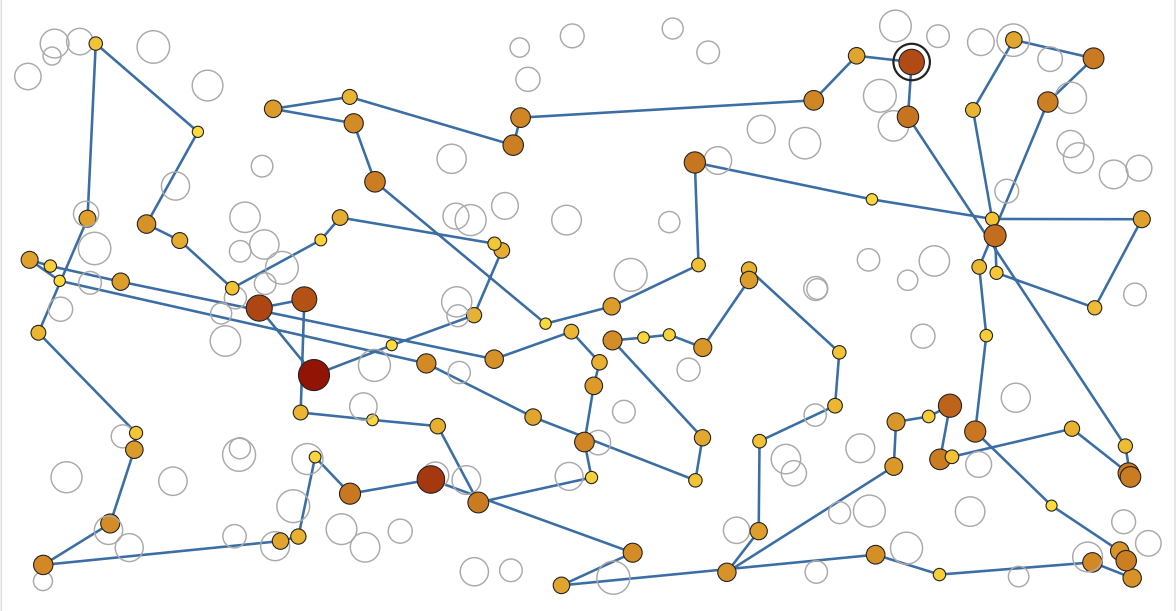
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
87, 189, 2, 171, 187, 141, 30, 97, 95, 127, 104, 129, 94, 69, 99, 174, 85, 8, 112, 91, 80, 116, 149, 63, 168,
83, 123, 152, 49, 60, 66, 39, 109, 62, 33, 9, 54, 134, 20, 158, 58, 84, 36, 186, 178, 59, 1, 169, 130, 88, 78,
153, 170, 159, 37, 147, 47, 93, 43, 106, 176, 150, 81, 50, 3, 45, 28, 180, 90, 132, 76, 71, 157, 115, 136, 19,
29, 55, 101, 46, 164, 177, 52, 16, 51, 35, 67, 144, 140, 53, 179, 42, 89, 107, 64, 57, 182, 0, 135, 121
```

Steepest LS, two-nodes exchange, random start, candidate moves K = 10

Best of 200 valid runs: run #140 with objective **86851**.



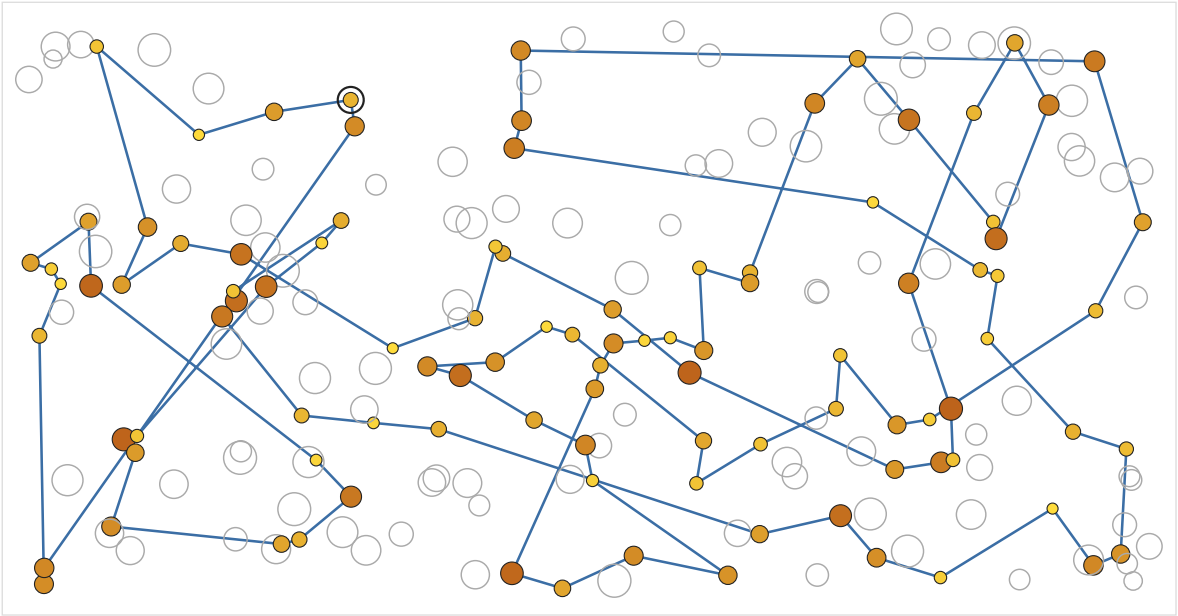
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
24, 136, 186, 112, 139, 157, 91, 71, 4, 67, 52, 33, 62, 164, 1, 147, 47, 50, 169, 178, 29, 55, 101, 3, 93, 113,
107, 153, 193, 78, 88, 57, 182, 114, 181, 96, 189, 187, 2, 171, 129, 69, 127, 97, 168, 60, 48, 80, 108, 152,
123, 132, 76, 51, 16, 35, 180, 23, 148, 162, 30, 141, 87, 98, 135, 73, 140, 53, 144, 185, 49, 149, 116, 63, 28,
45, 58, 84, 179, 177, 36, 131, 54, 9, 174, 176, 134, 158, 192, 151, 150, 106, 43, 42, 89, 130, 146, 170, 159,
184
```

Steepest LS, two-nodes exchange, random start, candidate moves K = 15

Best of 200 valid runs: run #33 with objective **85620**.



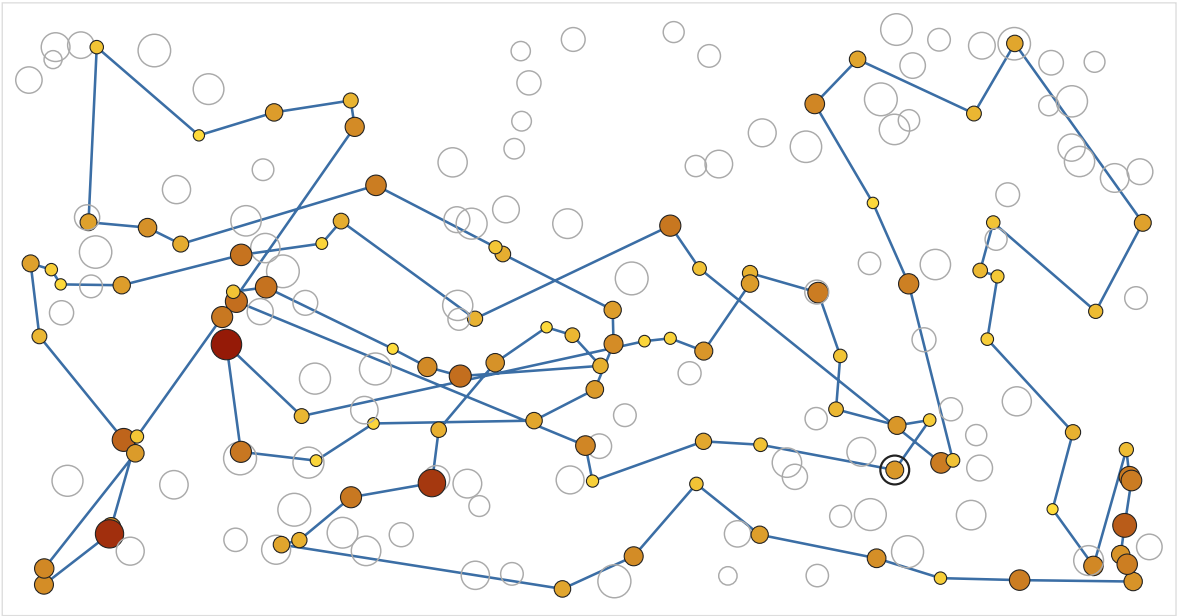
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
157, 91, 80, 48, 108, 49, 152, 100, 180, 35, 51, 16, 52, 99, 151, 42, 89, 43, 85, 178, 29, 101, 169, 1, 184,
136, 186, 9, 54, 33, 131, 36, 177, 179, 53, 140, 25, 114, 181, 192, 135, 73, 45, 102, 28, 185, 67, 144, 84, 58,
134, 176, 174, 150, 106, 47, 147, 55, 115, 112, 139, 164, 3, 50, 93, 130, 159, 153, 88, 107, 57, 182, 138, 158,
87, 141, 30, 160, 83, 123, 76, 132, 90, 97, 95, 127, 69, 171, 2, 189, 187, 11, 60, 116, 149, 63, 168, 104, 129,
71
```

Steepest LS, two-nodes exchange, random start, candidate moves K = 20

Best of 200 valid runs: run #149 with objective **84345**.



node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

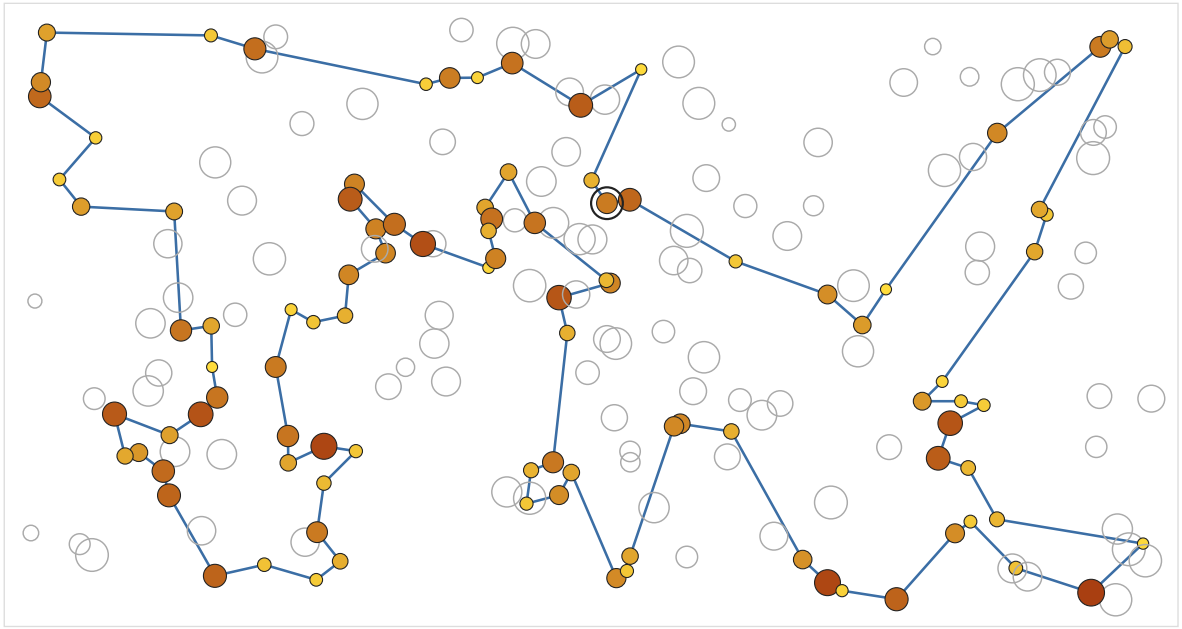
Best solution (node indices, from 0)

```
151, 106, 150, 176, 174, 8, 9, 54, 131, 36, 177, 30, 12, 161, 187, 141, 45, 140, 179, 52, 16, 51, 4, 152, 108,
60, 48, 80, 91, 157, 71, 160, 97, 69, 94, 104, 129, 127, 95, 168, 116, 149, 63, 49, 100, 132, 76, 35, 143, 33,
42, 89, 85, 164, 186, 136, 178, 29, 147, 47, 1, 3, 50, 93, 130, 107, 88, 159, 146, 170, 75, 153, 193, 78, 64,
57, 182, 158, 58, 181, 114, 171, 2, 189, 96, 87, 185, 67, 144, 53, 102, 28, 180, 90, 123, 83, 73, 135, 84, 134
```

Instance TSPB

Steepest LS, two-edges exchange, random start, baseline (all moves)

Best of 200 valid runs: run #175 with objective **50216**.



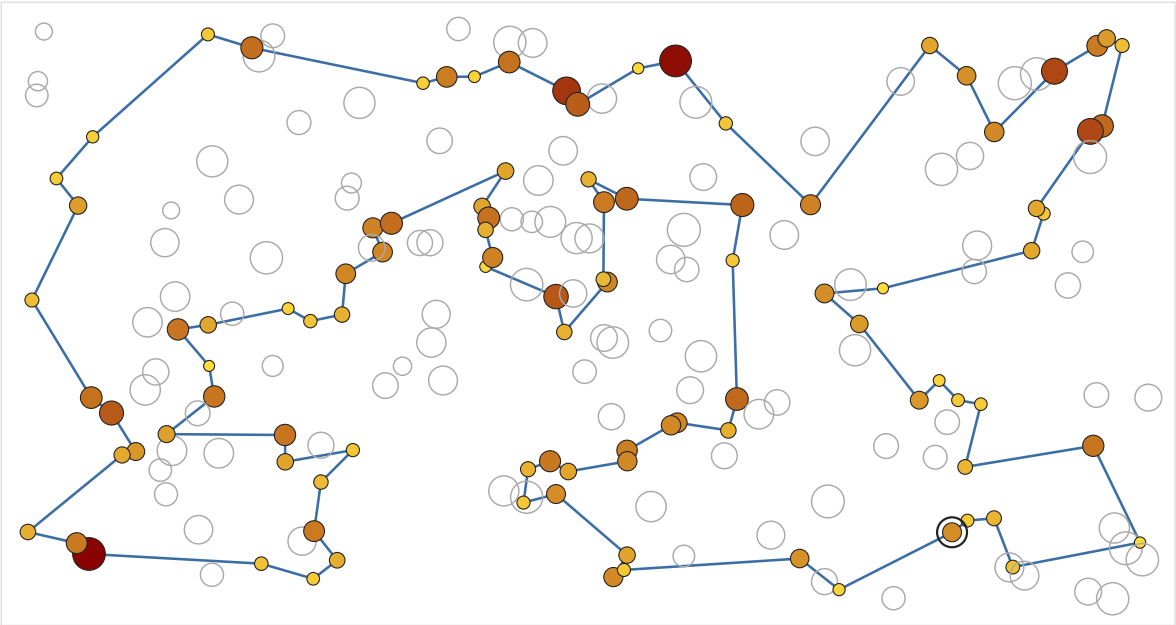
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
46, 86, 76, 162, 192, 54, 187, 81, 94, 105, 196, 114, 10, 127, 80, 129, 90, 52, 182, 153, 14, 110, 93, 2, 20,
74, 17, 171, 39, 99, 67, 24, 34, 179, 87, 82, 167, 51, 85, 59, 73, 151, 122, 3, 184, 117, 120, 133, 136, 172,
160, 69, 189, 183, 130, 191, 30, 21, 38, 47, 140, 154, 97, 155, 71, 141, 40, 60, 157, 45, 134, 175, 62, 36, 197,
29, 132, 103, 6, 180, 115, 170, 125, 177, 158, 145, 28, 149, 13, 146, 37, 116, 147, 121, 198, 96, 48, 156, 89,
72
```

Steepest LS, two-edges exchange, random start, candidate moves K = 5

Best of 200 valid runs: run #150 with objective **50600**.



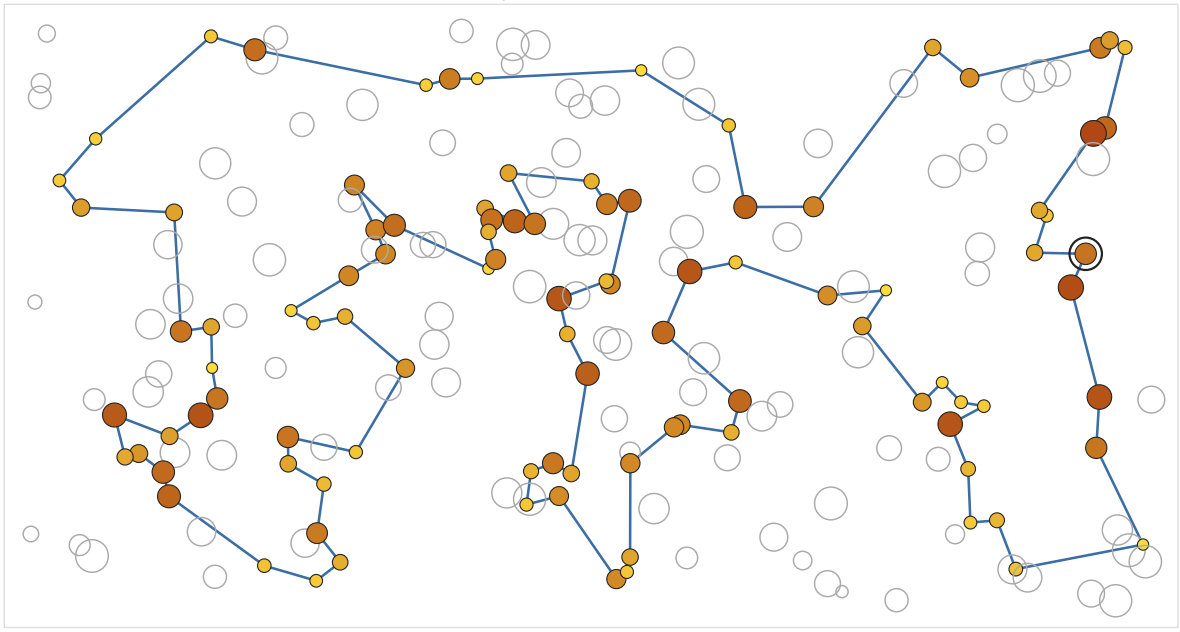
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
74, 171, 99, 87, 82, 179, 51, 85, 59, 73, 167, 181, 152, 34, 24, 67, 66, 76, 185, 86, 72, 46, 184, 3, 151, 122,
69, 160, 172, 136, 133, 120, 183, 30, 21, 38, 47, 140, 154, 125, 177, 170, 115, 6, 155, 71, 40, 60, 157, 45,
134, 175, 108, 174, 1, 132, 29, 103, 178, 65, 145, 28, 149, 116, 147, 121, 198, 96, 48, 43, 156, 89, 44, 57,
104, 190, 70, 187, 169, 81, 94, 105, 61, 139, 196, 114, 10, 54, 162, 192, 80, 127, 129, 90, 153, 118, 110, 2,
14, 20
```

Steepest LS, two-edges exchange, random start, candidate moves K = 10

Best of 200 valid runs: run #35 with objective **50088**.



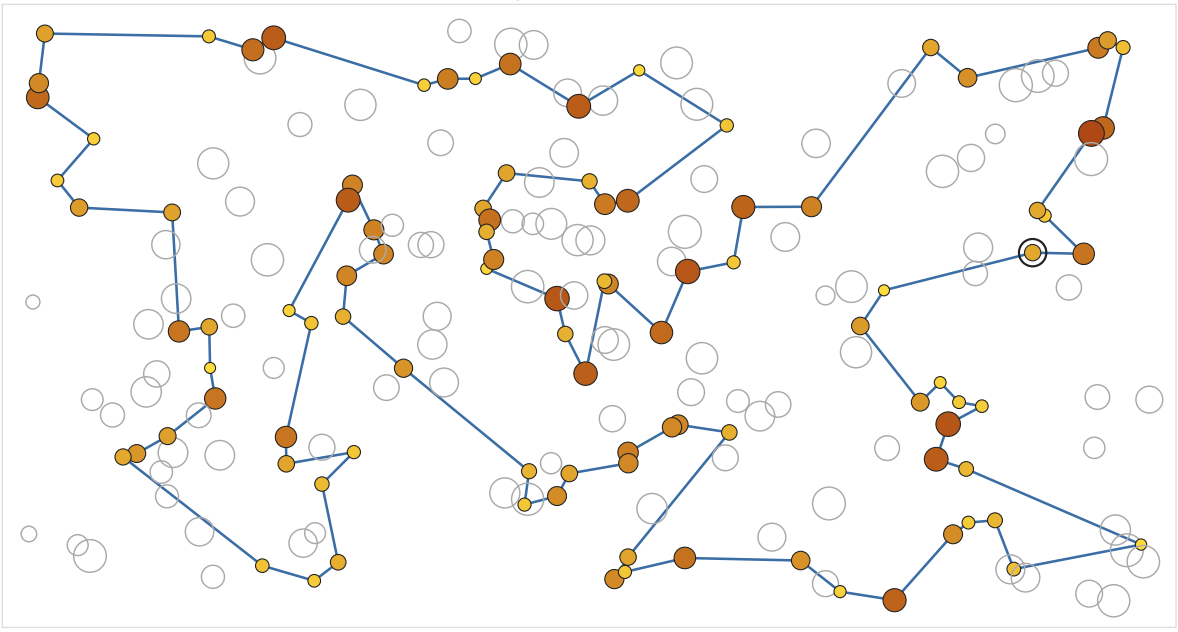
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
148, 143, 56, 118, 110, 2, 14, 20, 153, 52, 90, 129, 127, 80, 192, 54, 162, 76, 135, 27, 66, 67, 24, 34, 181,
179, 87, 82, 51, 85, 59, 73, 167, 98, 151, 122, 184, 3, 86, 46, 72, 120, 117, 8, 133, 136, 172, 160, 69, 183,
130, 30, 21, 38, 154, 140, 47, 64, 40, 155, 71, 60, 157, 45, 134, 175, 36, 197, 29, 132, 103, 6, 180, 115, 170,
125, 177, 158, 145, 28, 149, 116, 147, 121, 198, 96, 89, 57, 185, 104, 190, 70, 81, 94, 105, 61, 139, 196, 114,
10
```

Steepest LS, two-edges exchange, random start, candidate moves K = 15

Best of 200 valid runs: run #168 with objective **50315**.



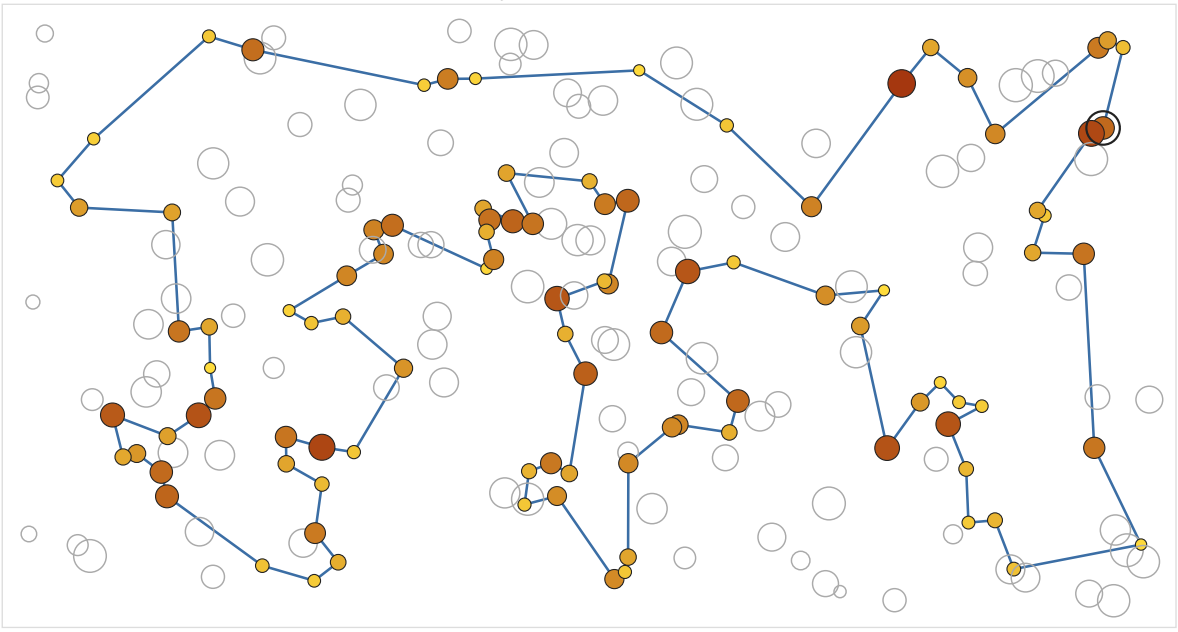
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
10, 148, 114, 196, 139, 61, 105, 94, 81, 70, 190, 104, 185, 76, 135, 27, 3, 184, 98, 151, 122, 69, 160, 172,
136, 133, 120, 72, 46, 86, 57, 89, 156, 48, 96, 198, 121, 42, 147, 116, 37, 146, 13, 149, 28, 145, 158, 177,
125, 170, 115, 6, 29, 132, 175, 134, 45, 60, 40, 71, 155, 140, 154, 191, 130, 30, 21, 38, 47, 64, 59, 85, 51,
167, 181, 152, 34, 24, 67, 179, 82, 87, 88, 99, 171, 17, 74, 20, 14, 2, 110, 153, 182, 52, 90, 129, 127, 80,
192, 54
```

Steepest LS, two-edges exchange, random start, candidate moves K = 20

Best of 200 valid runs: run #35 with objective **50381**.



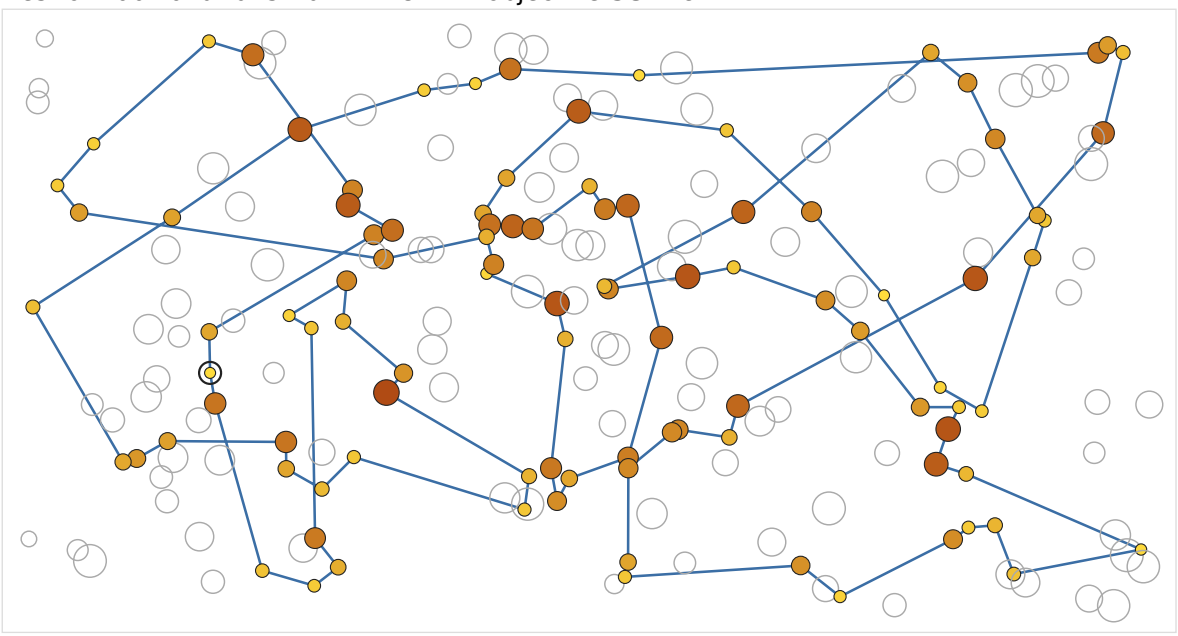
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
61, 139, 196, 114, 10, 148, 118, 110, 2, 14, 20, 153, 52, 90, 129, 127, 80, 150, 192, 54, 162, 76, 135, 27, 66,
67, 24, 34, 181, 179, 87, 82, 51, 85, 59, 73, 167, 98, 151, 122, 184, 3, 86, 46, 72, 120, 117, 8, 133, 136, 172,
160, 69, 183, 30, 21, 38, 154, 140, 47, 64, 40, 141, 155, 71, 60, 157, 45, 134, 175, 36, 197, 29, 132, 103, 6,
180, 115, 170, 125, 177, 158, 145, 28, 149, 116, 147, 121, 198, 96, 89, 57, 104, 194, 190, 70, 187, 81, 94, 105
```

Steepest LS, two-nodes exchange, random start, baseline (all moves)

Best of 200 valid runs: run #128 with objective **58746**.



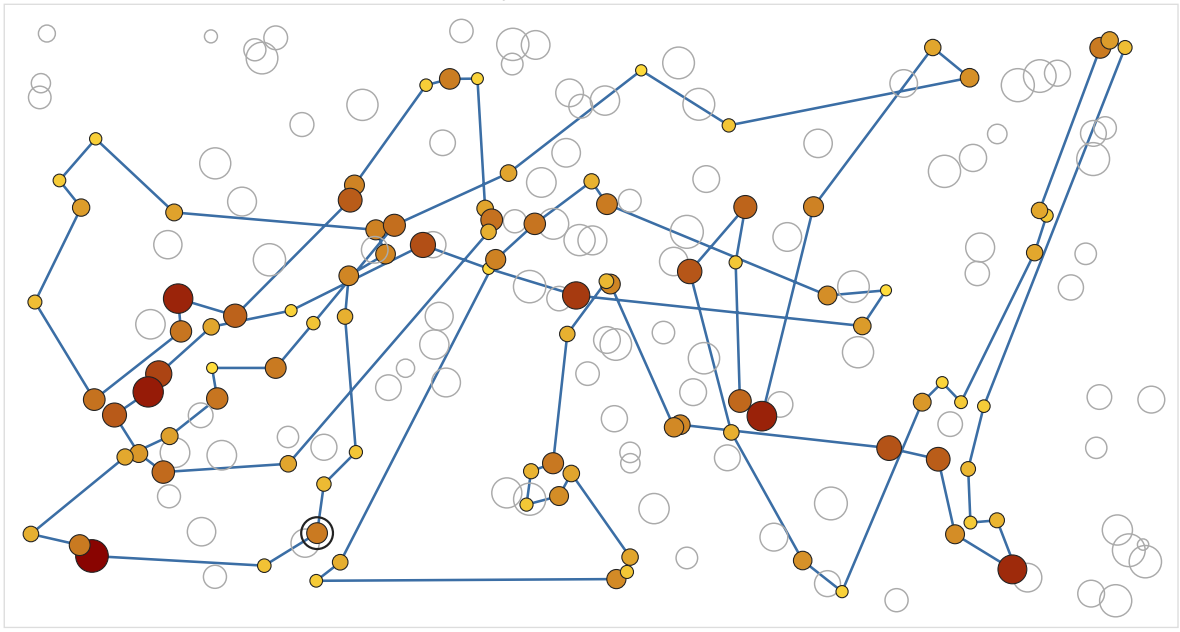
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
170, 125, 30, 183, 191, 130, 147, 116, 149, 28, 145, 21, 172, 160, 69, 122, 151, 73, 51, 167, 152, 27, 86, 46,
72, 117, 8, 136, 133, 120, 156, 57, 104, 54, 127, 90, 10, 114, 196, 187, 70, 190, 185, 184, 3, 135, 76, 162,
192, 80, 129, 52, 182, 153, 110, 2, 14, 20, 74, 171, 99, 87, 179, 181, 34, 24, 67, 66, 78, 61, 105, 94, 81, 89,
48, 96, 121, 50, 158, 65, 132, 29, 6, 155, 71, 60, 40, 85, 59, 33, 64, 47, 38, 154, 140, 157, 45, 134, 175, 115
```

Steepest LS, two-nodes exchange, random start, candidate moves K = 5

Best of 200 valid runs: run #19 with objective **63744**.



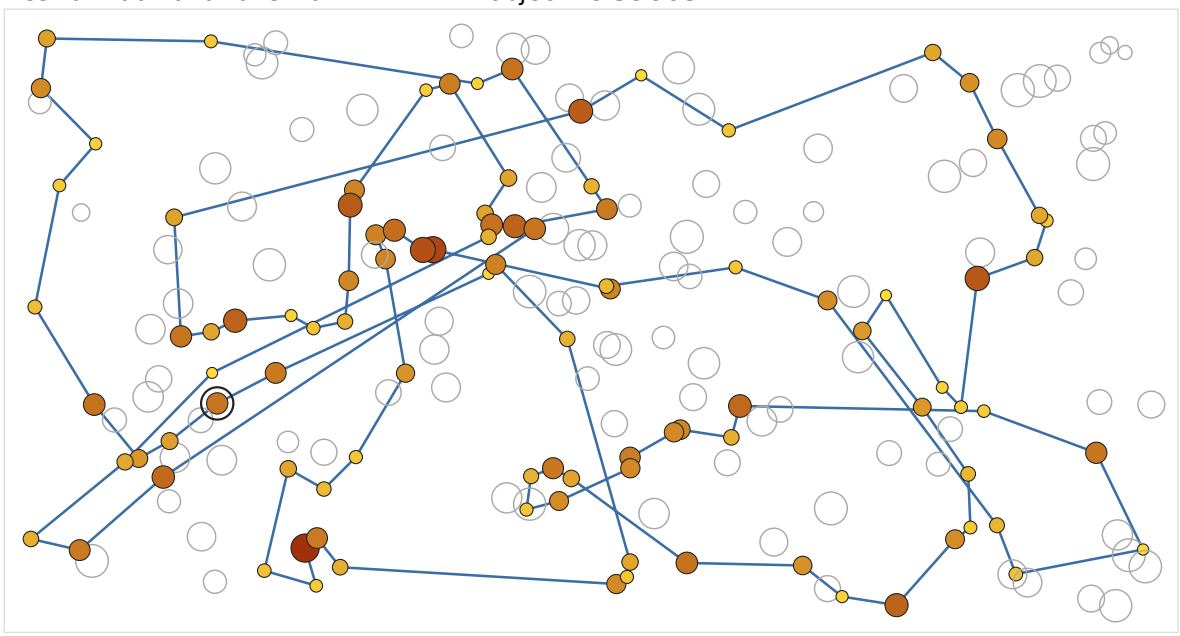
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
157, 175, 108, 174, 1, 132, 6, 115, 170, 97, 140, 183, 120, 89, 57, 70, 190, 104, 119, 66, 76, 185, 135, 67, 99,
171, 80, 127, 129, 10, 114, 196, 81, 94, 105, 90, 153, 20, 14, 2, 188, 74, 182, 150, 24, 34, 3, 184, 151, 73,
59, 85, 51, 167, 179, 87, 82, 134, 45, 160, 117, 72, 46, 162, 54, 192, 173, 69, 189, 154, 125, 19, 102, 103, 29,
197, 71, 172, 136, 133, 96, 198, 121, 130, 191, 26, 55, 177, 178, 65, 145, 28, 149, 158, 30, 21, 38, 47, 40, 60
```

Steepest LS, two-nodes exchange, random start, candidate moves K = 10

Best of 200 valid runs: run #124 with objective **59665**.



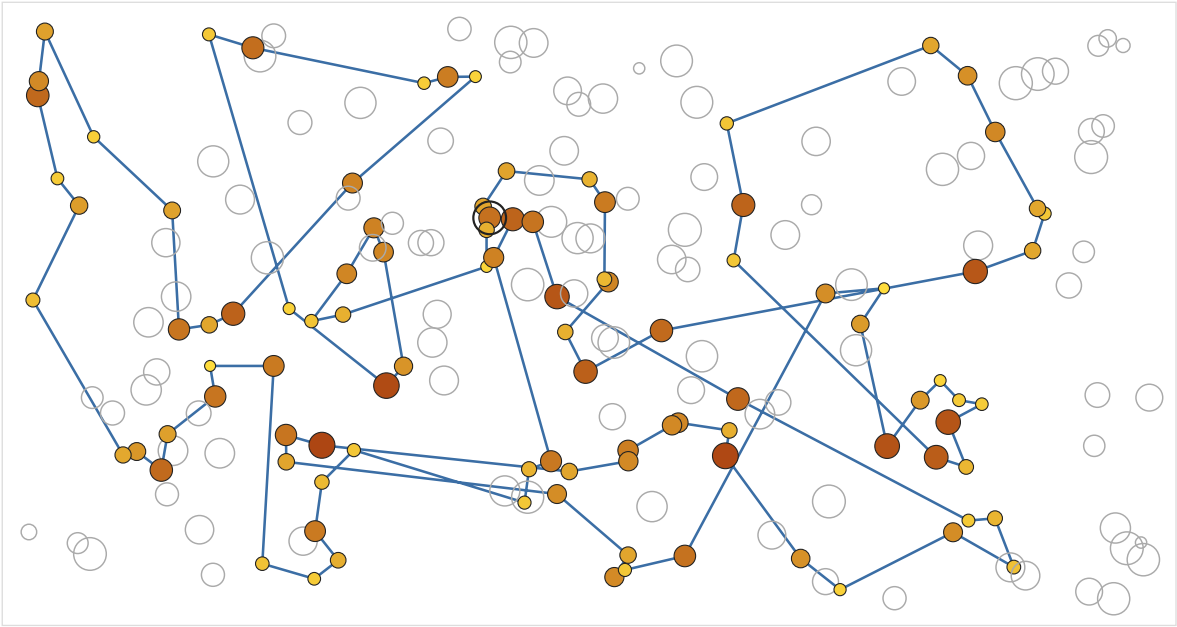
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
115, 6, 29, 178, 65, 28, 149, 146, 37, 116, 96, 48, 72, 46, 8, 117, 197, 174, 1, 132, 170, 172, 136, 133, 120,
198, 121, 130, 191, 38, 47, 140, 154, 26, 125, 177, 158, 156, 89, 57, 190, 70, 187, 196, 114, 10, 78, 129, 127,
54, 192, 80, 153, 20, 74, 17, 171, 99, 88, 167, 73, 59, 85, 51, 181, 152, 34, 24, 67, 66, 90, 118, 110, 2, 14,
162, 76, 184, 3, 168, 189, 183, 30, 21, 64, 40, 60, 71, 175, 134, 142, 157, 45, 82, 87, 179, 151, 160, 69, 97
```

Steepest LS, two-nodes exchange, random start, candidate moves K = 15

Best of 200 valid runs: run #89 with objective **58873**.



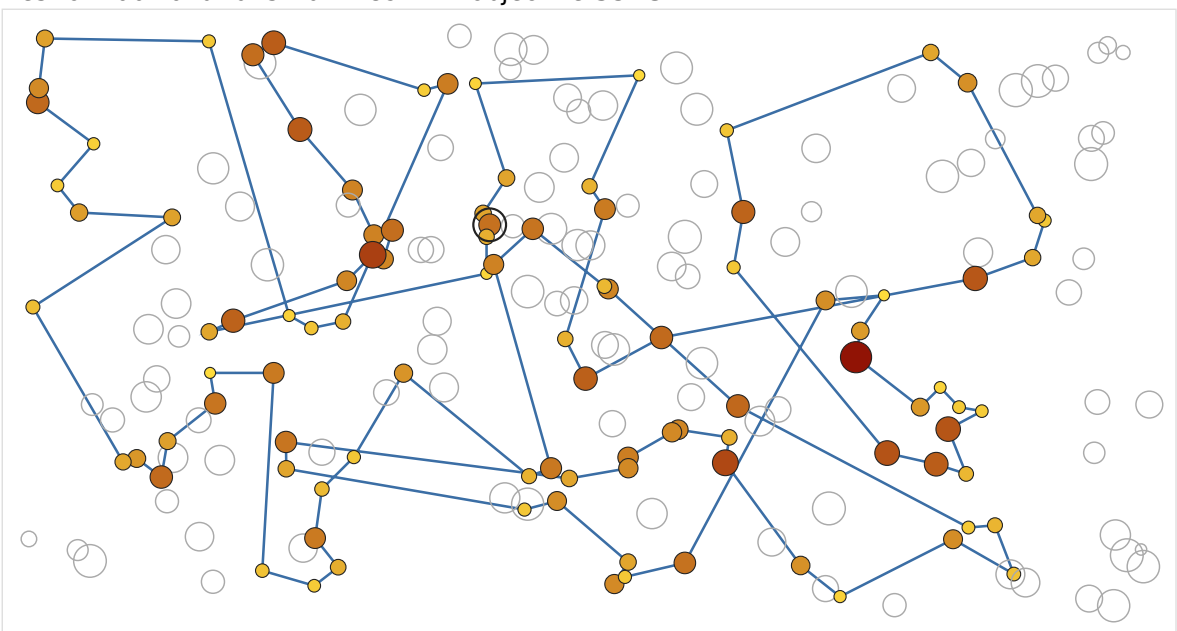
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
136, 172, 69, 47, 140, 38, 30, 21, 64, 33, 154, 116, 147, 121, 198, 96, 130, 26, 125, 177, 158, 149, 37, 146,
13, 28, 145, 65, 132, 29, 197, 6, 115, 170, 97, 175, 134, 45, 157, 60, 40, 85, 59, 73, 160, 8, 117, 122, 66, 20,
14, 2, 74, 171, 99, 31, 67, 24, 34, 152, 181, 167, 141, 155, 71, 51, 179, 82, 87, 88, 162, 54, 192, 150, 80,
127, 129, 90, 52, 153, 182, 76, 185, 57, 190, 70, 187, 196, 114, 10, 78, 27, 98, 151, 3, 184, 46, 72, 120, 133
```

Steepest LS, two-nodes exchange, random start, candidate moves K = 20

Best of 200 valid runs: run #89 with objective **59787**.



node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
136, 172, 69, 125, 26, 38, 176, 21, 183, 30, 130, 50, 147, 42, 121, 198, 47, 140, 154, 116, 37, 146, 13, 149,
28, 145, 158, 65, 132, 29, 197, 6, 115, 170, 97, 175, 134, 45, 157, 60, 40, 64, 59, 73, 160, 117, 184, 3, 66,
20, 14, 2, 74, 171, 99, 31, 67, 24, 34, 152, 181, 167, 155, 71, 85, 51, 179, 82, 87, 88, 162, 54, 192, 106, 80,
127, 129, 90, 52, 153, 182, 150, 76, 185, 57, 190, 70, 196, 114, 10, 78, 27, 98, 151, 46, 72, 89, 96, 120, 133
```

Source code: https://github.com/CodeSencor/evolutionary-computation

Conclusions

TSPA: the best average objective is 76199.32 (Steepest LS, two-edges exchange, random start, baseline (all moves)); the fastest method is “Steepest LS, two-edges exchange, random start, candidate moves K = 5” with 2.206 ms per run on average, 9.65× faster than its baseline with an average objective +13.35 % different. **TSPB:** the best average objective is 52463.99 (Steepest LS, two-edges exchange, random start, candidate moves K = 15); the fastest method is “Steepest LS, two-edges exchange, random start, candidate moves K = 5” with 1.883 ms per run on average, 10.83× faster than its baseline with an average objective +2.99 % different.

The main task was to improve the speed of steepest local search by using candidate moves.As we can see in instance B it even improves the performance. We ran the code with 3 different seeds,and for each of them in instance B the performance improved with the use of candidate moves. We think it means thatmaybe the few steepest first steps may seem the best but lead into a local optimum which is impossibleto escape later. Regarding the types of exchanges, the random start lead to two nodes exchange lead to notoriously poor results.